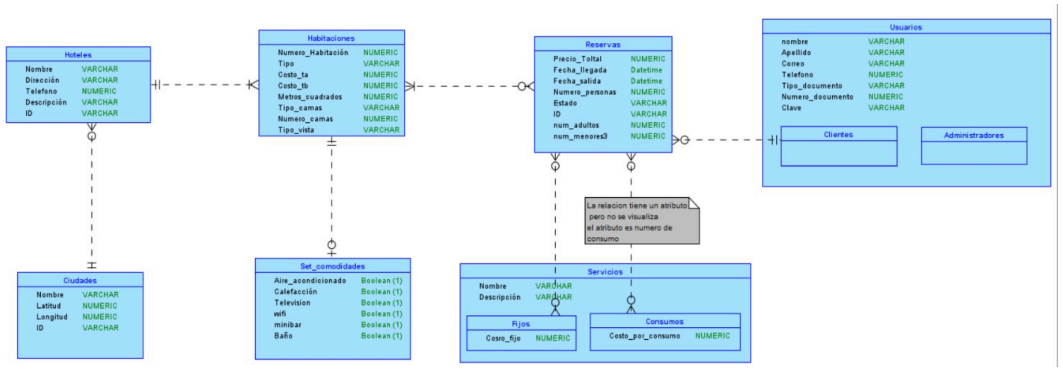
## Análisis y modelo conceptual:

- ➢ Proponga un modelo conceptual en E/R que describa las entidades del modelo de datos para la aplicación que se quiere desarrollar. Para ello use Data Modeler.




## Diseño de la base de datos:

- ➢ Desarrolle el modelo de datos relacional correspondiente al modelo conceptual E/R propuesto. Este modelo debe quedar descrito en la plantilla de Excel disponible para ello en BN. Debe justificar la selección de las tablas a través del proceso de transformación de E/R a relacional visto en clase (algoritmo de Chen)

Ciudades

| id_ciudad | nombre | latitud | longitud |
|---|---|---|---|
| PK,SA | NN, SA | NN, SA | NN, SA |
| | | | |
| | | | |
| | | | |

Hoteles

| id_hotel | nombre | direccion | telefono | descripcion | id_ciudad |
|---|---|---|---|---|---|
| PK, SA | UA, NN | NN, UA | NN, US | NN, UA | NN, FK Ciudades |
| | | | | | |
| | | | | | |
| | | | | | |

Habitaciones

| id_hotel | numero | tipo | costo_ta | costo_tb | metros_cuadrados | tipo_camas | numero_camas | tipo_vista | id_habitacion |
|---|---|---|---|---|---|---|---|---|---|
| NN, FK Hoteles | UA, NN | NN, CK[Suite Junior, Habitacion Doble, Suite Presidencial] | NN , CK[mayor a 0] | NN, CK[mayor a 0] | NN,CK[mayor a 0] | NN, CK[King, Queen, Doble, Single] | NN, CK[mayor a 0] | NN, CK[Mar, Monumentos, Piscina, estandar] | PK |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |

Comodidades

| id_habitacion | aire_acondicionado | calefaccion | television | wifi | minibar | banio |
|---|---|---|---|---|---|---|
| PK, FK Habitaciones.numero_habitacion | NN, CK[True, false] | NN, CK[True, false] | NN, CK[True, false] | NN, CK[True, false] | NN, CK[True, false] | NN, CK[True, false] |
| | | | | | | |
| | | | | | | |
| | | | | | | |

Habitacion por Reserva

| id_habitacion | id_reserva |
|---|---|
| PK, FK Habitaciones | PK, FK Usuarios |
| | |
| | |
| | |

Reservas

| precio_total | fecha_llegada | fecha_salida | num_personas_adultas | num_ninio_menor_3 | num_personas | estado | Id_reserva | id_usuario |
|---|---|---|---|---|---|---|---|---|
| SA, NN | UA, NN | UA, NN | UA, NN | UA, NN | NN,DD | SA, NN | PK, SA | NN, FK Clientes |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |

Servicios

| nombre | descripción | id_servicio |
|---|---|---|
| UA, NN, ND | UA, NN, ND | SA, PK |
| | | |
| | | |
| | | |

Servicios por Reserva

| id_servicio | id_reserva |
|---|---|
| PK, FK Servicios Fijos | PK, FK Reservas |
| | |
| | |
| | |

Clientes

| id_usuario |
| --- |
| PK, FK usuarios |
| |
| |
| |

Administradores

| id_usuario |
| --- |
| PK, FK usuarios |
| |
| |
| |

Servicios Fijos

| id_servicio | costo_fijo |
| --- | --- |
| PK, FK servicios | UA, NN |
| | |
| | |
| | |

Consumos por Reserva

| id_reserva | id_servicio | consumos |
| --- | --- | --- |
| PK, FK reservas | PK, FK servicios por consumo | NN, UA |
| | | |
| | | |
| | | |

Servicios por Consumo

| id_servicio | Costo_por_consumo |
| --- | --- |
| PK, FK servicios | UA, NN |
| | |
| | |
| | |

Usuarios

| id_usuario | nombre | apellido | correo | telefono | tipo_documento | numero_documento | clave |
| --- | --- | --- | --- | --- | --- | --- | --- |
| SA, PK | UA, NN | UA, NN | UA, NN, ND, CK(tener un valido) | NN, ND, UA, CK(longitud de 10) | NN, UA, CK(C.C., Pasaporte) | NN, ND, UA | NN, UA, CK(longitud de 10, contenga un símbolo, mayusculas, minusculas y numeros) |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |

Según el algoritmo de Chen se pasó las entidades a tablas y los atributos son sus respectivas columnas en las siguientes relaciones:

- Ciudad, Hoteles, Habitaciones, Comodidades, Reservas, Servicios, Servicios fijos, Servicios por consumo, Usuarios, Clientes y Administradores.

Adicional a esto en las relaciones de uno a muchos, con el extremo univalor obligatorio, se pasó la pk del extremo univalor como atributo del extremo multivalor, con una restricción de no nulos, esto se visualiza en las siguientes relaciones siendo el extremo univalor obligatorio y el segundo el multivalor opcional respectivamente:

- Ciudades a Hoteles
- Hoteles a Habitaciones
- Clientes a Reservas

Por otro las relaciones de muchos a muchos se hizo una conversión en una nueva tabla con el fk de ambas tablas que es la llave compuesta de la tabla, esto se realizó en las siguientes relaciones:

- Habitaciones a Reservas

Adicional a esto en aquellas tablas que tenían el tipo de relación anterior con atributos, se añadieron estos atributos a las tablas nuevas:

- Reservas a Servicios fijos
- Reservas a Servicios por consumo

Además, la conversión de una relación de uno a uno donde uno es obligatorio se pasó el pk del no obligatorio como fk, con restricción de no nulo a la tabla donde es obligatorio, esto se realizó en la siguiente relación:

- Habitaciones a set de comodidades

De igual manera la conversión de herencias, al ser herencia disjunta completa, se modeló usando relaciones de 1 a muchos tanto para las herencias de servicios como usuarios.

Por último en las tablas se agregaron las siguientes restricciones según la necesidad del campo:

- PK: llave de la tabla
- FK: llave que viene de otra tabla
- NN: no se permiten valores nulos
- DD: valores que derivan de otros
- SA: valor asignado por el sistema
- US: valor asignado por el usuario
- CK: valores que complen con algunas caracteristicas

➢ Determine el nivel de normalización en que se encuentra su modelo y realice las transformaciones necesarias del mismo para llevarlo a forma normal Boyce-Codd (FNBC). Adjunte a la entrega la verificación de las formas normales vistas en clase para cada relación, las transformaciones sobre las mismas y el modelo de datos relacional final resultante.

- **Ciudades(id_ciudad, nombre, latitud, altitud)**

C(I,N,A,O)

**Dependencias funcionales:**
I → N, A, O

**primos** = {I}
**no primos** = {N, A, O}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene llaves compuestas.

Ciudades

| id_ciudad | nombre | latitud | longitud |
|---|---|---|---|
| PK,SA | NN, SA | NN, SA | NN, SA |
| | | | |
| | | | |
| | | | |

- **Hoteles(id_hotel, nombre, dirección, teléfono, descripción, id_ciudad)**

H(I, N, D, T, E, C)

**Dependencias funcionales:**
I → N, D, T, E, C

**primos** = {I}
**no primos** = {N, D, T, E, C}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene llaves compuestas.

Hoteles

| id_hotel | nombre | direccion | telefono | descripcion | id_ciudad |
|---|---|---|---|---|---|
| PK, SA | UA, NN | NN, UA | NN, US | NN, UA | NN, FK Ciudades |
| | | | | | |
| | | | | | |
| | | | | | |

- **Habitaciones(id_habitación, numero, tipo, costo_ta, costo_tb, metros_cuadrados, tipo_camas, numero_camas, tipo_vista, id_hotel)**

S(I, N, T, A, B, M, C, K, V, H)

**Dependencias funcionales:**
I → N, T, A, B, M, C, K, V, H
H, N → T, A, B, M, C, K, V

**primos** = {I}
**no primos** = {N, T, A, B, M, C, K, V, H}

llave candidata = I

Se encuentra en 2° nivel de normalización, ya que no tiene listas como valor, ni tiene dependencias parciales, sin embargo si tiene dependencia transitiva.

Habitaciones

| id_hotel | numero | tipo | costo_ta | costo_tb | metros_cuadrados | tipo_camas | numero_camas | tipo_vista | id_habitacion |
|---|---|---|---|---|---|---|---|---|---|
| NN, FK Hoteles | UA, NN | NN, CK [Suite Junior, Habitacion Doble, Suite Presidencial] | NN , CK [mayor a 0] | NN, CK [mayor a 0] | NN,CK [mayor a 0] | NN, CK [King, Queen, Doble, Sengle] | NN, CK [mayor a 0] | NN, CK [Vista, Monumentos, Piscina, estandar] | PK |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |

S(I, H, N)
S´(N, T, A, B, M, C, K, V, H)

Con esta modificación la tabla ya se encuentra en BCNF, ya que no hay transitividad y no existen llaves compuestas.

Identificacion_habitaciones

| id_hotel | numero | id_habitacion |
|---|---|---|
| NN, FK Habitaciones | NN, FK Habitaciones | PK |
| | | |
| | | |
| | | |

Informacion_habitaciones

| id_hotel | numero | tipo | costo_ta | costo_tb | metros_cuadrados | tipo_camas | numero_camas | tipo_vista |
|---|---|---|---|---|---|---|---|---|
| PK, FK Hoteles | PK | NN, CK [Suite Junior, Habitacion Doble, Suite Presidencial] | NN , CK [mayor a 0] | NN, CK [mayor a 0] | NN,CK [mayor a 0] | NN, CK [King, Queen, Doble, Single] | NN, CK [mayor a 0] | NN, CK [Mar, Monumentos, Piscina, |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |

- **Comodidades(id_habitacion, aire_acondicionado, calefacción, televisión, wifi, minibar, banio)**

R(I, A, C, T, W, M, B)

**Dependencias funcionales:**
I → A, C, T, W, M, B

**primos** = {I}
**no primos** = {A, C, T, W, M, B}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene llaves compuestas.

Comodidades

| id_habitacion | aire_acondicionado | calefaccion | television | wifi | minibar | banio |
|---|---|---|---|---|---|---|
| PK, FK Habitaciones.numero_habitacion | NN, CK [True, False] | NN, CK [True, False] | NN, CK [True, False] | NN, CK [True, False] | NN, CK [True, False] | NN, CK [True, False] |
| | | | | | | |
| | | | | | | |
| | | | | | | |

- **Reservas(id_reserva, precio_total, fecha_llegada, fecha_salida, num_personas_adultas, num_personas_menor_3, num_personas, estado, id_usuario)**

R(I, P, L, S, A, N, K, E, U)

**Dependencias funcionales**
I → P, L, S, A, N, K, E, U

**primos** = {I}
**no primos** = {P, L, S, A, N, K, E, U}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene llaves compuestas.

Reservas

| precio_total | fecha_llegada | fecha_salida | num_personas_adultas | num_ninio_menor_3 | num_personas | estado | Id_reserva | id_usuario |
|---|---|---|---|---|---|---|---|---|
| SA, NN | UA, NN | UA, NN | UA, NN | UA, NN | NN,DD | SA, NN | PK, SA | NN, FK$_{clientes}$ |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |

- **Servicios(id_servicio, nombre, descripción)**

S(I, N, D)

**Dependencias funcionales:**
I → N, D

**primos** = {I}
**no primos** = {N, D}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene llaves compuestas.

Servicios

| nombre | descripción | id_servicio |
|---|---|---|
| UA, NN, ND | UA, NN, ND | SA, PK |
| | | |
| | | |
| | | |

- **Servicios por reserva(id_servicio, id_reserva)**

S(I, R)

**primos** = {I, R}

llave candidata = I, R

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene problemas con la llave compuesta.

Servicios por Reserva

| id_servicio | id_reserva |
|---|---|
| PK, FK $_{Servicios\ fijos}$ | PK, FK $_{Reservas}$ |
| | |
| | |
| | |

- **Servicios Fijos(id_servicios, costo_fijo)**

F(I, C)

**Dependencias funcionales:**
I → C

**primos** = {I}
**no primos** = {C}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene llaves compuestas.

Servicios Fijos

| id_servicio | costo_fijo |
|---|---|
| PK, FK Servicios | UA, NN |
| | |
| | |
| | |

- **Consumos por reserva(id_servicio, id_reserva, consumos)**

S(I, R, C)

**Dependencias funcionales:**
I, R → C

**primos** = {I, R}
**no primos** = {C}

llave candidata = I, R

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene problemas con la llave compuesta.

Consumos por Reserva

| id_reserva | id_servicio | consumos |
|---|---|---|
| PK, FK Reservas | PK, FK Servicios por consumo | NN, UA |
| | | |
| | | |
| | | |

- **Servicios por Consumo(id_servicios, costo_por_consumo)**

G(I, C)

**Dependencias Funcionales:**
I → C

**primos** = {I}
**no primos** = {C}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene llaves compuestas.

Servicios por Consumo

| id_servicio | Costo_por_consumo |
|---|---|
| PK, FK Servicios | UA, NN |
| | |
| | |
| | |

- **Usuarios(id_usuario, nombre, apellido, correo, telefono, tipo_documento, numero_documento, clave)**

U(I, N, A, C, T, M, D, V)

**Dependencias funcionales:**
I → N, A, C, T, M, D, V
M, D → N, A

**primos** = {I}
**no primos** = { N, A, C, T, M, D, V}

llave candidata = I

Se encuentra en 2° nivel de normalización, ya que no tiene listas como valor, ni tiene dependencias parciales, sin embargo si tiene dependencia transitiva.

Usuarios

| id_usuario | nombre | apellido | correo | telefono | tipo_documento | numero_documento | clave |
|---|---|---|---|---|---|---|---|
| SA, PK | UA, NN | UA, NN | UA, NN, ND, CK(tener url valida) | NN, ND, UA, CK(longitud de 10) | NN, UA, CK(C.C, Pasaporte) | NN, ND, UA | NN, UA, CK(longitud de 10, contenga un símbolo, mayusculas, minusculas y numeros) |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |

U(C, T, M, D, V)
U´(M, D, A, N)

Con esta modificación la tabla ya se encuentra en BCNF, ya que no hay transitividad y no existen llaves compuestas.

Identificacion_usuarios

| tipo de documento | numero_documento | nombre | apellido |
|---|---|---|---|
| NN, UA, CK[C.C, Pasaporte] | NN, ND, UA | UA, NN | UA, NN |
| | | | |
| | | | |
| | | | |

Informacion_usuarios

| id_usuario | correo | telefono | tipo de documento | numero_documento | clave |
|---|---|---|---|---|---|
| SA, PK | UA, NN, ND, CK[tener un valido] | NN, ND, UA, CK[longitud de ...] | NN, UA, CK[C.C, Pasaporte] | NN, ND, UA | NN, UA, CK[longitud de 10, contenga un minuscula, mayuscula, ...] |
| | | | | | |
| | | | | | |
| | | | | | |

- **Habitaciones por reserva(id_habitacion, id_reserva)**

M(H, R)

**primos** = {H,R}

llave candidata = H, R

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y no tiene problemas con la llave compuesta.

Habitacion por Reserva

| id_habitacion | id_reserva |
|---|---|
| PK, FK habitaciones | PK, FK Usuarios |
| | |
| | |
| | |

- **Clientes(id_usuario)**

C(I)

**primos** = {I}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y ni tiene llaves compuestas.

Clientes

| id_usuario |
|---|
| PK, FK Usuarios |
| |
| |
| |

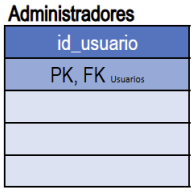
- **Administradores(id_usuario)**

C(I)

**primos** = {I}

llave candidata = I

Se encuentra en el BCNF, ya que no tiene listas como valor, ni tiene dependencias parciales, ni transitivas y ni tiene llaves compuestas.




**Escenarios de prueba:**

- Diseñe escenarios que les permitirán probar los RF (ver secciones de Requerimientos funcionales del documento marco del caso de estudio (RF1-RF7 y RFC1-RFC5) y asegurar la corrección y calidad de los datos en la base de datos. A continuación, damos algunos ejemplos para probar la corrección y calidad del modelo en la base de datos:
    i. Pruebas de unicidad de tuplas: para una de las tablas, describa inserciones o borrados de tuplas una PK conocida y nueva. Enseguida trate de insertar una tupla 2, con la misma PK que la tupla 1. 4
    ii. Pruebas de integridad con FK: trate de insertar una tupla 1 que tenga una FK que se encuentra en la tabla referenciada. De igual manera trate de insertar una tupla 1 que tenga una FK que no se encuentra en la tabla referenciada. Haga las pruebas de inserción para cada caso
    iii. Pruebas de integridad de acuerdo con restricciones de chequeo: trate de insertar tuplas que violan las restricciones de chequeo establecidas
- Para probar la corrección de la aplicación, incluya pruebas habituales de funcionalidad que revisen las reglas de negocio. No olvide considerar los roles de usuario, casos exitosos, y casos de falla.

RF1 – Registrar ciudad Para este requerimiento, se inserta una fila en la tabla Ciudades definiendo el atributo id_ciudad y el nombre. Si el administrador intenta registrar una ciudad con un id_ciudad que ya existe, el sistema bloquea la operación por duplicidad. Una vez guardado el registro, la ciudad queda disponible para ser vinculada a cualquier hotel de la cadena.

RF2 – Registrar hotel Al registrar un hotel, se crea una entrada en la tabla Hoteles con el atributo id_hotel y se vincula obligatoriamente al atributo id_ciudad de una ciudad previamente existente. Si se intenta ingresar un id_ciudad que no figura en la tabla referenciada, el sistema impide el registro para mantener la integridad de la ubicación. Tras el registro exitoso, el hotel queda habilitado para configurar su inventario.

RF2.5 – Registrar servicio Para este proceso, se crea un registro en la tabla Servicios definiendo el atributo id_servicio y el tipo_cobro. El sistema utiliza una restricción de chequeo para asegurar que el tipo_cobro indique correctamente si se aplica una sola vez o por noche. Si se intenta registrar un servicio con un costo negativo, la base de datos bloquea la inserción para garantizar la calidad de los datos financieros.

RF3 – Registrar habitaciones para un hotel Cuando se registra un tipo de habitación, se añade una fila a Habitaciones definiendo los atributos costo_ta y costo_tb junto con la capacidad. El sistema vincula estos datos al id_hotel correspondiente para que cada sede tenga su propio catálogo. Si la información ingresada viola alguna restricción de capacidad o de costo mínimo, el sistema rechaza la tupla para evitar errores en la oferta comercial.

RF4 – Editar habitaciones En este requerimiento, el administrador modifica atributos como costo_ta o dotacion directamente en la tabla Habitaciones. El sistema asegura que esta edición se aplique únicamente al registro identificado por su id_hotel y numero_habitacion. Si el cambio es exitoso, los nuevos valores se verán reflejados en las futuras consultas sin alterar la información de las demás sedes de la cadena.

RF5 – Creación de cuenta de cliente Para crear una cuenta, se inserta la información en la tabla Usuarios y se crea el rol en la tabla Clientes mediante el atributo id_usuario. El sistema verifica que el atributo correo cumpla con el formato estándar de una dirección electrónica. Si el id_usuario ya está registrado para otra persona, la base de datos bloquea el nuevo registro para proteger la identidad de los usuarios existentes.

RF6 – Gestión de cuenta de cliente Cuando el cliente gestiona su cuenta, accede a la tabla Usuarios para modificar atributos como telefono o clave. El sistema sobreescribe el valor antiguo por el nuevo y guarda el cambio de manera inmediata. Si el usuario intenta modificar atributos que el sistema define como protegidos, la operación se cancela para mantener la consistencia de la información personal.

RF7 – Realizar reserva de habitación Al realizar una reserva, se crea un registro en la tabla Reservas con el atributo estado en 'Confirmada' y se guardan las fechas en los atributos fecha_llegada y fecha_salida. Simultáneamente, se inserta una fila en Habitacion_por_Reserva para bloquear el cupo. Si la habitación seleccionada ya tiene un registro en ese mismo intervalo de fechas, el sistema impide la creación de la reserva por falta de disponibilidad.

RF7 – Gestionar la reserva por parte del cliente Cuando un cliente desea cancelar, el sistema consulta el atributo fecha_llegada en la tabla Reservas. Si faltan al menos tres días para el inicio de la estadía, el atributo estado de la reserva cambia de 'Confirmada' a 'Cancelada'. Si el intento ocurre fuera de ese plazo, el sistema bloquea el cambio de este atributo y la reserva permanece activa con el cobro correspondiente.

RFC1 – Consultar habitaciones disponibles Para esta consulta, el sistema escanea la tabla Habitaciones y cruza los datos con Habitacion_por_Reserva. El resultado muestra

únicamente las habitaciones que no tienen registros de ocupación en los atributos de fecha solicitados por el usuario. Si todas las habitaciones de un tipo están vinculadas a reservas activas en ese rango, el sistema devuelve una lista vacía para ese criterio.

RFC2 – Consultar reservas a través del tiempo Esta consulta extrae información de la tabla Reservas utilizando el atributo fecha_llegada para agrupar los registros por temporadas. El administrador puede filtrar los resultados mediante los atributos id_hotel e id_ciudad. El éxito de la prueba se da cuando el sistema genera un reporte histórico que permite identificar los meses con mayor número de registros en estado 'Confirmada'.

RFC3 – Consultar la cantidad de huéspedes por fechas y ciudad Para este reporte, el sistema realiza una suma de los atributos adultos y niños presentes en la tabla Reservas. La consulta filtra los registros que coinciden con el atributo id_ciudad de los hoteles involucrados y el rango de fechas pedido. El resultado final muestra el volumen total de personas alojadas, permitiendo al administrador conocer la ocupación real por ciudad.

RFC4 – Consultar el ingreso total por hotel por rango de fechas En este escenario, el sistema suma los costos de la tabla Habitaciones y los registros asociados en Consumos_por_Reserva para cada reserva finalizada. La consulta utiliza el atributo id_hotel para agrupar los montos generados durante un año específico. El reporte es exitoso cuando se presenta el acumulado de ingresos reflejando la actividad financiera real de cada sede.

RFC5 – Consultar los servicios y tipos de habitación más populares Este requerimiento analiza la frecuencia de aparición de los atributos id_servicio en Consumos_por_Reserva y el tipo de habitación en Habitaciones. El sistema cuenta cuántas veces se ha solicitado cada elemento en las reservas procesadas. El resultado permite al administrador visualizar las preferencias de los clientes basándose estrictamente en los registros históricos almacenados en las tablas.